

Resumen

La creación de una API RESTful es una habilidad fundamental para cualquier desarrollador web moderno. En este artículo, exploraremos cómo implementar la operación PUT en nuestro CRUD, permitiéndonos actualizar registros de usuarios de manera efectiva. Aprenderemos a manejar rutas dinámicas, procesar parámetros y actualizar datos en un archivo JSON, todo mientras identificamos posibles problemas y mejores prácticas.

¿Cómo implementar la operación PUT en una API RESTful?

Continuando con nuestro desarrollo CRUD, ahora nos enfocaremos en la operación PUT, que nos permite actualizar información de usuarios existentes. Esta operación es esencial en cualquier aplicación que maneje datos persistentes.

Para implementar correctamente un endpoint PUT, necesitamos:

1. Definir una ruta dinámica que acepte un identificador.
2. Recibir y procesar los datos actualizados.
3. Encontrar el registro específico a modificar.
4. Aplicar los cambios y guardar la información actualizada.

Comencemos definiendo nuestra ruta en la aplicación:

```
app.put('/users/:id', (req, res) => {  
  // Obtenemos el ID del usuario desde los parámetros de la ruta  
  const userId = parseInt(req.params.id, 10);  
  
  // Obtenemos los datos actualizados del cuerpo de la petición  
  const updateUser = req.body;  
  
  // Lógica para actualizar el usuario  
  // ...  
});
```

¿Cómo manejar la lectura y actualización de datos?

Una vez definida nuestra ruta, necesitamos implementar la lógica para leer el archivo de usuarios, encontrar el usuario específico, actualizarlo y guardar los cambios:

```
app.put('/users/:id', (req, res) => {  
  const userId = parseInt(req.params.id, 10);  
  const updateUser = req.body;  
  
  // Leemos el archivo de usuarios  
  fs.readFile(userFilePath, 'utf8', (error, data) => {  
    if (error) {  
      return res.status(500).json({ error: "Error con conexión de datos" });  
    }  
  
    // Parseamos los datos JSON
```

```

let users = JSON.parse(data);

// Actualizamos el usuario que coincida con el ID
users = users.map(user => {
  return user.id === userId ? { ...user, ...updateUser } : user;
});

// Guardamos los cambios en el archivo
fs.writeFile(userFilePath, JSON.stringify(users, null, 2), (err) => {
  if (err) {
    return res.status(500).json({ error: "Error al actualizar el usuario"
  });
  }

  // Respondemos con el usuario actualizado
  res.json(updateUser);
});
});
});
});

```

En este código:

1. Convertimos el ID de la URL a un número entero con `parseInt()`.
2. Extraemos los datos actualizados del cuerpo de la petición.
3. Leemos el archivo de usuarios existente.
4. Utilizamos el método `map()` para recorrer todos los usuarios y actualizar solo el que coincida con el ID proporcionado.
5. Guardamos los cambios en el archivo.
6. Respondemos con el usuario actualizado.

¿Qué problemas pueden surgir al actualizar registros?

Durante la implementación de la operación PUT, es importante estar atento a posibles problemas:

Duplicidad de IDs: Si existen múltiples registros con el mismo ID en nuestra base de datos, todos serán actualizados cuando intentemos modificar uno específico. Esto puede causar inconsistencias graves en los datos.

Por ejemplo, si tenemos dos usuarios con ID=1 y actualizamos uno de ellos, ambos registros se modificarán:

```

// Antes de la actualización
[
  { id: 1, name: "Jane", email: "jane@example.com" },
  { id: 2, name: "Bob", email: "bob@example.com" },
  { id: 1, name: "Alice", email: "alice@example.com" }
]

// Después de actualizar el usuario con ID=1
[

```

```
[ { id: 1, name: "John Smith", email: "john2@example.com" },  
  { id: 2, name: "Bob", email: "bob@example.com" },  
  { id: 1, name: "John Smith", email: "john2@example.com" }  
]
```

Falta de validaciones: Es crucial implementar validaciones para:

- Verificar que el ID existe antes de intentar actualizar.
- Validar que los datos enviados cumplen con el formato esperado.
- Asegurar que no existan IDs duplicados en la base de datos.

¿Cómo probar nuestra implementación PUT?

Para probar nuestra implementación, podemos utilizar herramientas como Postman:

1. Crear una nueva solicitud PUT.
2. Establecer la URL con el ID del usuario a actualizar (por ejemplo, `http://localhost:3000/users/1`).
3. Configurar el cuerpo de la solicitud con los datos actualizados en formato JSON:

```
{  
  "name": "John Smith",  
  "email": "john2@example.com"  
}
```

4. Enviar la solicitud y verificar la respuesta.
5. Comprobar que los cambios se hayan aplicado correctamente en nuestro archivo de usuarios.

Importante: Siempre verifica que los cambios se hayan aplicado correctamente y solo a los registros deseados.

La implementación de la operación PUT es fundamental para cualquier API RESTful completa. Con una correcta validación y manejo de errores, podemos asegurar que nuestros datos se mantengan consistentes y que las actualizaciones se apliquen de manera precisa.

¿Has implementado alguna vez una API con operaciones CRUD? Comparte tu experiencia en los comentarios.

Excelente trabajo 🙌 como aporte la validación de los campos de esta manera `if (name) users[userIndex].name = name; if (email) users[userIndex].email = email;` cuando son muchos campos se vuelve más complicada, por lo que usar el spread operator facilita esta tarea, ya que sí tienen propiedades con el mismo nombre, la última sobrescribe a la anterior. `{ ...user, ...updatedUser }` por lo que no es necesario validar dato por dato.

```

app.put('/users/:id', (req, res) => {
  const userId = parseInt(req.params.id, 10);
  const updatedUser = req.body;
  const {name, email} = updatedUser;
  const regex_email = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  const regex_name = /^[a-zA-Z\s]{3,}$/;

  // Check if all required fields are present
  if (!name) {
    return res.status(400).json({ error: 'Missing required field: name' });
  } else if (!email) {
    return res.status(400).json({ error: 'Missing required field: email' });
  };

  //Data validation
  if (!updatedUser || Object.keys(updatedUser).length === 0) {
    return res.status(400).json({ error: 'No user data provided' });
  }

  if (!name || !regex_name.test(name)) {
    return res.status(400).json({ error: 'User name must be at least 3 characters long and contain only letters' });
  }

  if (!email || !regex_email.test(email)) {
    return res.status(400).json({ error: 'Invalid email format' });
  }

  fs.readFile(usersFilePath, 'utf-8', (err, data) => {
    if (err) {
      return res.status(500).json({ error: 'Error with data connection' });
    }
    let users = JSON.parse(data);
    let userFound = false;
    users = users.map(user => user.id === userId ? (userFound = true, {...user, ...{name,email}}) : user);
    if (!userFound) {
      return res.status(404).json({ error: 'User not found' });
    }
    fs.writeFile(usersFilePath, JSON.stringify(users, null, 2), (err) => {
      if (err) {
        return res.status(500).json({ error: 'Error saving user' });
      }
      res.json({name,email});
    });
  });
});

```

